3.2、图像几何变换

[OpenCV: Geometric Transformations of Images]
https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html

这个教程非常实用，主要讲解如何对图像进行缩放、平移、旋转、仿射变换、透视变换。这些操作在图像处理、计算机视觉中非常常见（比如校正倾斜的文档、做数据增强等）。主要就是先get变换矩阵，再cv.warpAffine()

1. 缩放（Scaling / Resizing）

最简单的变换，就是改变图像大小。核心函数：cv.resize()

```
# 方法1：使用缩放因子（推荐）
res1 = cv.resize(img, None, fx=2, fy=2, interpolation=cv.INTER_CUBIC)   # 放大2倍

# 方法2：直接指定目标尺寸
height, width = img.shape[:2]
res2 = cv.resize(img, (2*width, 2*height), interpolation=cv.INTER_CUBIC)
```

插值方法（interpolation）很重要：

- cv.INTER_NEAREST：最近邻（最快，但锯齿明显）
- cv.INTER_LINEAR：双线性（默认，速度与质量平衡）
- cv.INTER_CUBIC：双三次（放大时质量最好，但较慢）
- cv.INTER_AREA：缩小图像时推荐使用（抗锯齿效果好）
  小贴士：缩小用 INTER_AREA，放大用 INTER_CUBIC 或 INTER_LINEAR。

2. 平移（Translation）

把图像向左/右、上/下移动。
变换矩阵（2×3）：

$$M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$$

其中 t_x 是水平偏移（正数向右），t_y 是垂直偏移（正数向下）。
核心函数：cv.warpAffine()

```
rows, cols = gray.shape
M = np.float32([[1, 0, 100],   # 向右移动100像素
```

```
                [0, 1, 50]])   # 向下移动50像素

dst = cv.warpAffine(gray, M, (cols, rows))  # 注意：dsize 是（宽度，高度）
cv.imshow('Translated', dst)
cv.waitKey(0)
cv.destroyAllWindows()
```

注意：warpAffine 的第三个参数是输出图像尺寸 (width, height)，千万别写反！

3. 旋转（Rotation）

绕某个点旋转图像，还可以同时缩放
核心函数：

- cv.getRotationMatrix2D(center, angle, scale) → 生成旋转矩阵
- 再用 cv.warpAffine() 应用

```
rows, cols = gray.shape
center = ((cols-1)/2.0, (rows-1)/2.0)   # 图像中心

M = cv.getRotationMatrix2D(center, 90, 1)   # 旋转90度，缩放1倍
dst = cv.warpAffine(gray, M, (cols, rows))
```

参数说明：

- center：旋转中心（通常用图像中心）
- angle：旋转角度（正数为逆时针）
- scale：缩放因子（>1 放大，<1 缩小）

4. 仿射变换（Affine Transformation）

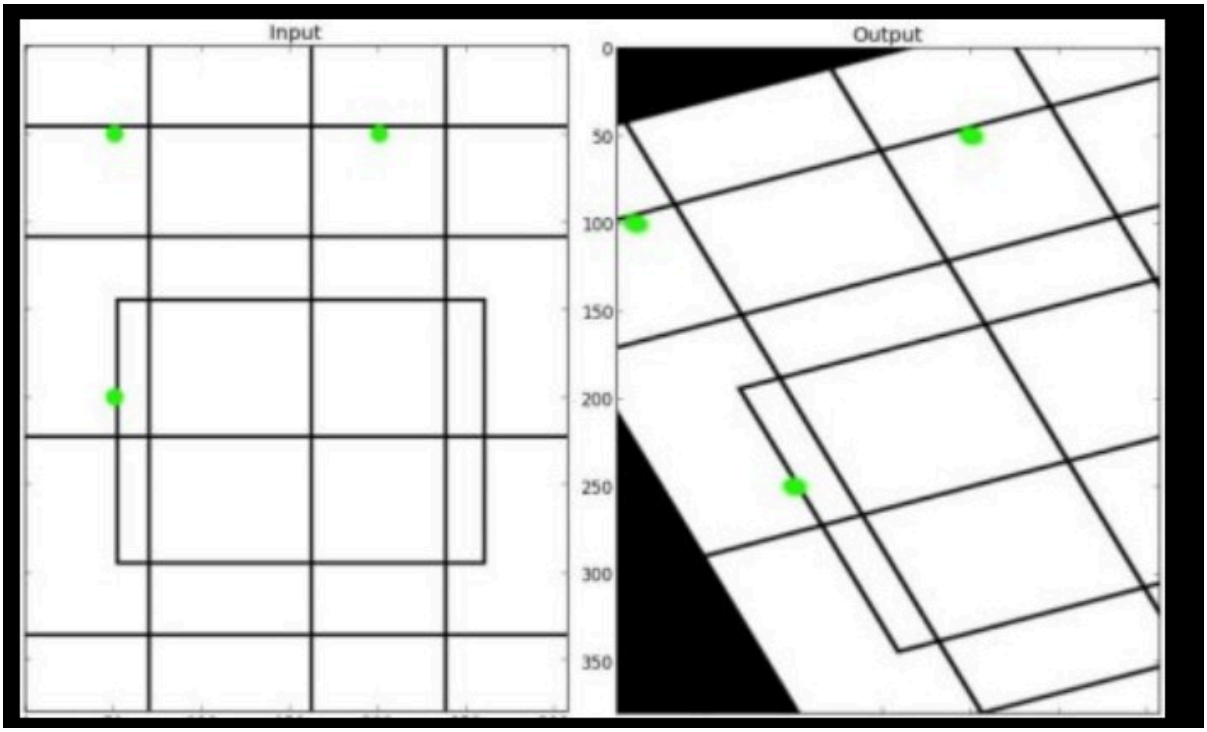
可以做非均匀缩放、平移、旋转、剪切（平行线保持平行）。
需要3对对应点来计算变换矩阵。

```
img = cv.imread('drawing.png')   # 建议用有明显形状的图片测试
rows, cols, ch = img.shape

# 输入图像上的3个点
pts1 = np.float32([[50,50], [200,50], [50,200]])
# 希望变换后的对应3个点
pts2 = np.float32([[10,100], [200,50], [100,250]])

M = cv.getAffineTransform(pts1, pts2)
dst = cv.warpAffine(img, M, (cols, rows))

# 显示对比
plt.subplot(121), plt.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)),
plt.title('Input')
plt.subplot(122), plt.imshow(cv.cvtColor(dst, cv.COLOR_BGR2RGB)),
plt.title('Affine')
plt.show()
```



5. 透视变换（Perspective Transformation）

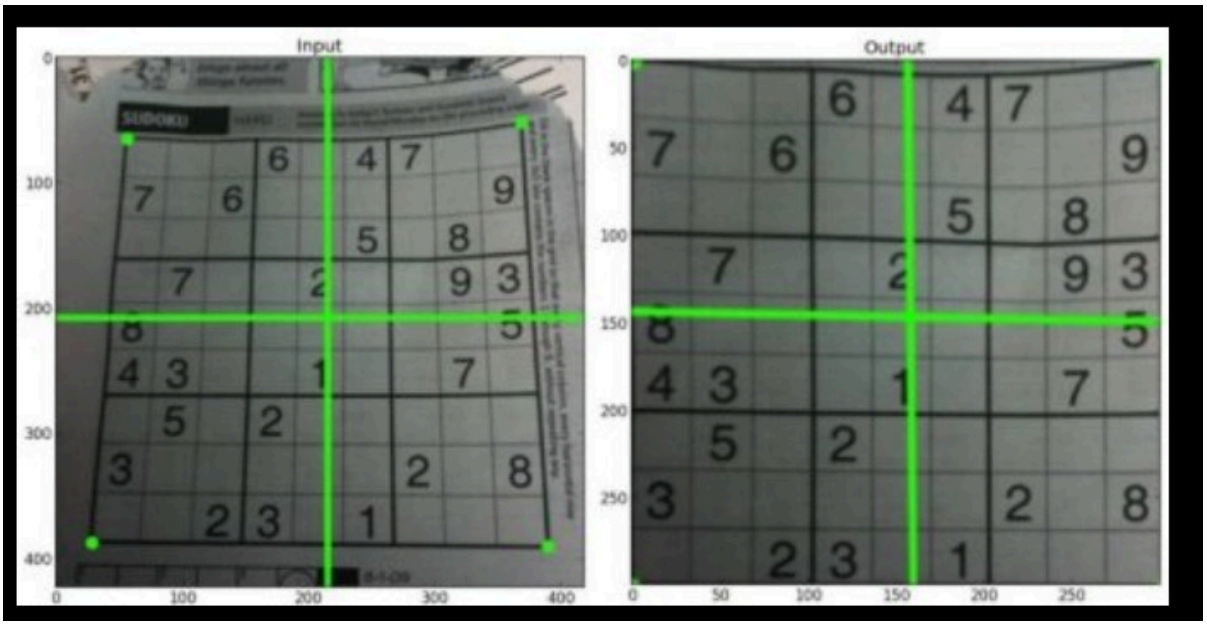
最强大的变换，能模拟相机视角变化（纠正倾斜的书、名片、棋盘等），直线仍然是直线。
需要4对对应点（其中任意3点不共线）。

```python
img = cv.imread('sudoku.png')   # 典型的透视例子
rows, cols, ch = img.shape

# 输入图像四个角点（大概位置）
pts1 = np.float32([[56,65], [368,52], [28,387], [389,390]])
# 希望变换成矩形（正视图）
pts2 = np.float32([[0,0], [300,0], [0,300], [300,300]])

M = cv.getPerspectiveTransform(pts1, pts2)
dst = cv.warpPerspective(img, M, (300, 300))   # 输出固定为300x300

plt.subplot(121), plt.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)),
plt.title('Input')
plt.subplot(122), plt.imshow(cv.cvtColor(dst, cv.COLOR_BGR2RGB)),
plt.title('Perspective')
plt.show()
```



总结对比

变换类型	需要点数	矩阵维度	核心函数	适用场景
缩放	-	-	cv.resize()	改变图像尺寸
平移	-	2×3	warpAffine + 手动矩阵	移动位置
旋转	-	2×3	getRotationMatrix2D + warpAffine	旋转图像
仿射变换	3对	2×3	getAffineTransform + warpAffine	剪切、倾斜

变换类型	需要点数	矩阵维度	核心函数	适用场景
透视变换	4对	3×3	getPerspectiveTransform + warpPerspective	纠正透视畸变